Camunda Message Event

In this session we will look at what is message event and how it functions ?.

Most processes require communication, which can be represented in BPMN by means of the message event. Message events are events which reference a named message. A message has a name and a payload.

Message event is always directed at a single recipient.

1. Start message event

If you want a process or event sub process to start when a message is received, use a start message event in your business process definition or inside your event sub process.

This effectively allows us to select the right start event from a set of alternative start events using the message name.

Incoming messages can originate from a message event in a process.

When deploying a process definition with one or more message start events, the following considerations apply:

- The name of the message start event must be unique across a given process definition, i.e., a process definition must not have multiple message start events with the same name. The engine throws an exception upon deployment of a process definition in case two or more message start events reference the same message or if two or more message start events reference messages with the same message name.
- The name of the message start event must be unique across all deployed process definitions. The engine throws an exception upon deployment of a process definition in case one or more message start events reference a

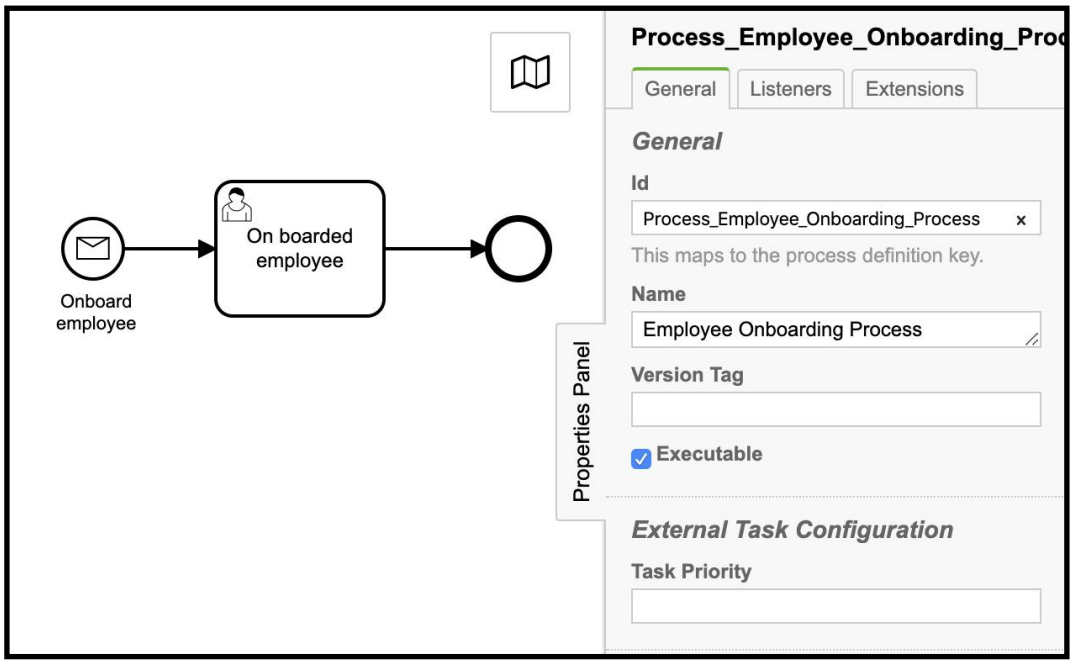
message with the same name as a message start event already deployed by a different process definition.

- Process versioning: Upon deployment of a new version of a process definition, the message subscriptions of the previous version are canceled. This is also the case for message events that are not present in the new version.
- Message start events are only supported in top-level processes. Message start events are not supported in embedded sub processes.

For example, employee on-boarding process to start when a record for each new employee is created in your HR system. When the record is created, the system sends an event to Business Process Manager. BPM captures the event and starts the follow-on steps for each new employee such as setting up the necessary space and computer equipment, requesting and creating a security badge.

1. Process definition

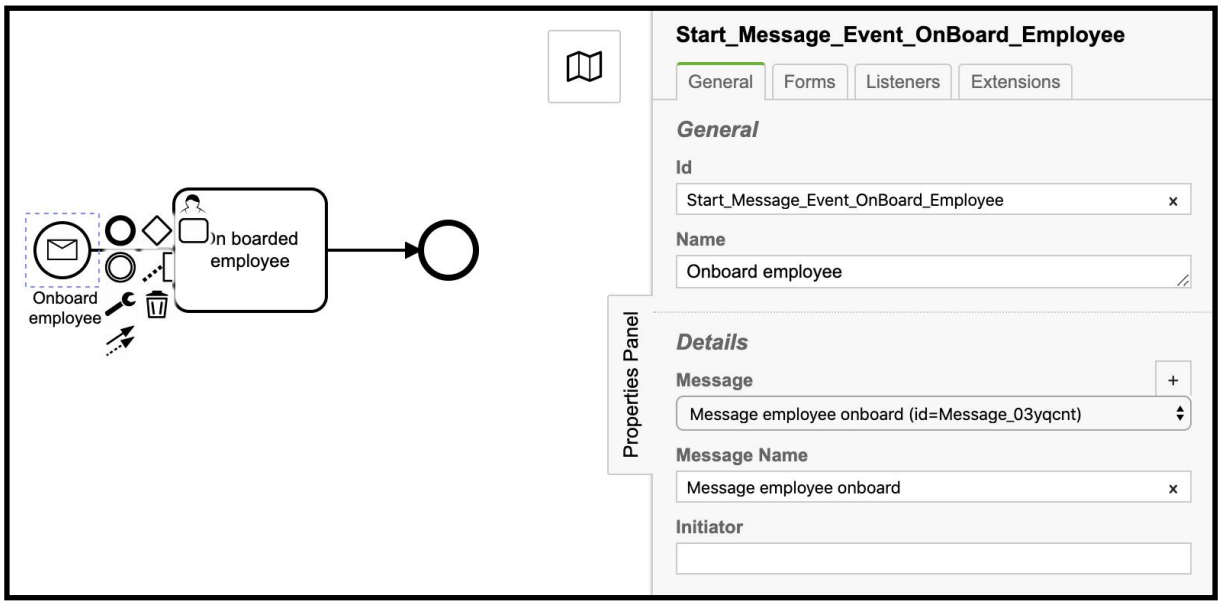
To demonstrate message start event we will use eclipse Camunda project because we will trigger message start event by unit test case.



Id: Process_Employee_Onboarding_Process

Name: Employee Onboarding Process

1.1 Start message event:



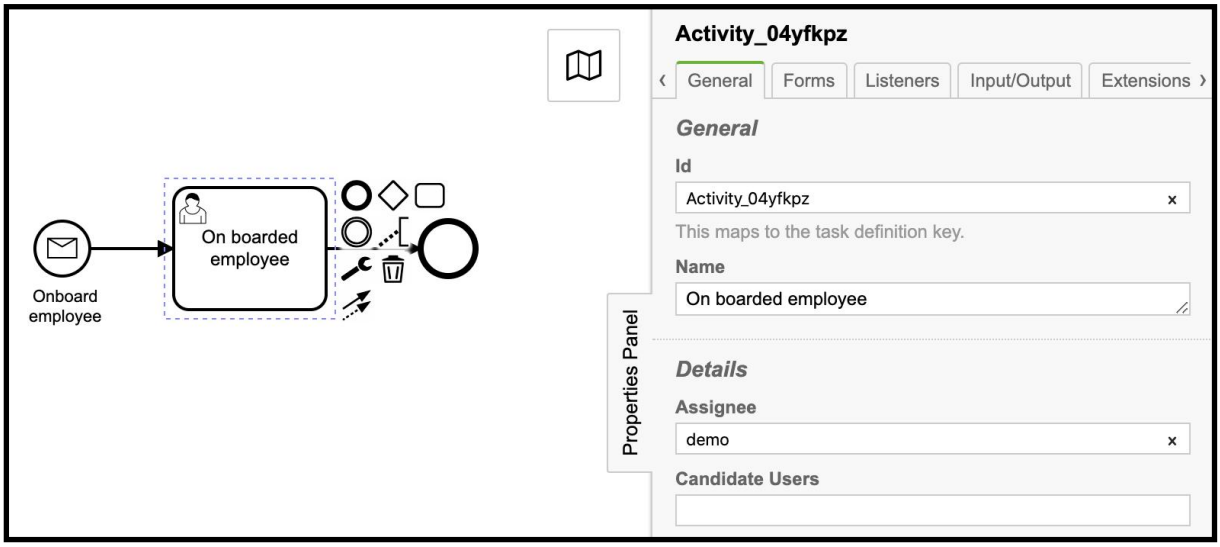
Id: Start_Message_Event_OnBoard_Employee

Name: Onboard employee

Message: Message employee onboard (id=Message_03yqcnt)

Message name: Message employee onboard

1.2 User task: On boarded employee



Id: Activity_04yfkpz

Name: On boarded employee

Assignee: demo

1.3 CamundaBpmProcessApplication.java

```
package com.camunda.training;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Logger;

import org.camunda.bpm.application.PostDeploy;
import org.camunda.bpm.application.ProcessApplication;
import org.camunda.bpm.application.impl.ServletProcessApplication;
import org.camunda.bpm.engine.ProcessEngine;
import org.camunda.bpm.engine.RuntimeService;
import org.camunda.bpm.engine.runtime.ProcessInstance;

/**
 * Process Application exposing this application's resources the process engine.
 */
@ProcessApplication
public class CamundaBpmProcessApplication extends ServletProcessApplication {

	private final Logger LOGGER =
Logger.getLogger(CamundaBpmProcessApplication.class.getName());

	private static final String MESSAGE_ID = "Message employee onboard";

	/**
	 * In a @PostDeploy Hook you can interact with the process engine and
access the
	 * processes the application has deployed.
	 */
	@PostDeploy
	public void onDeploymentFinished(ProcessEngine processEngine) {
		RuntimeService runtimeService = processEngine.getRuntimeService();
		LOGGER.info("Runtime service started");

		Map<String, Object> variableMap = new HashMap<String, Object>();
		variableMap.put("firstName", "John");
		variableMap.put("lastName", "Sinha");
		variableMap.put("employeeId", "E 1230");
```

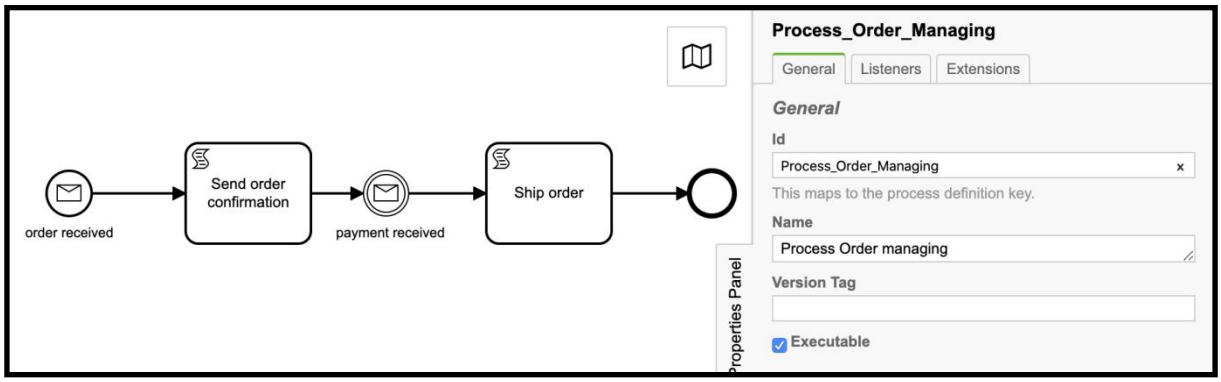
```
                ProcessInstance processInstance =
runtimeService.startProcessInstanceByMessage(MESSAGE_ID, variableMap);
                LOGGER.info("Process instance started : process instance id : " +
processInstance.getId());
        }

}
```

On deployment of process application, process instance will be initiated and user task On boarded employee will get created.

2. Message intermediate catch event

When a token arrives at the message intermediate catching event it will wait there until a message with the proper name arrives. As already described, the message must be handed into the engine via the appropriate API calls.

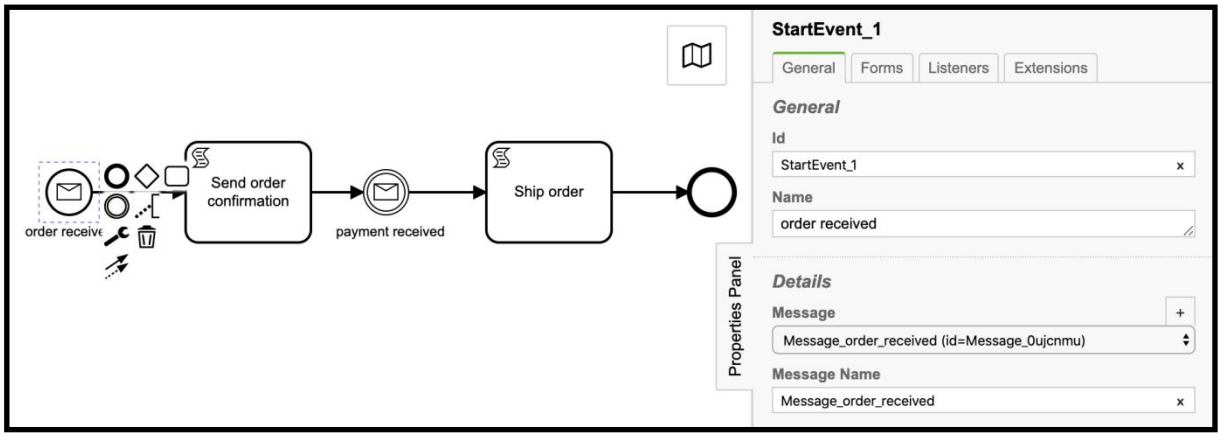
Resume a process flow (message intermediate catch event) when the message arrives.



Id: Process_Order_Managing

Name: Process Order managing

2.1 Message start event: order received



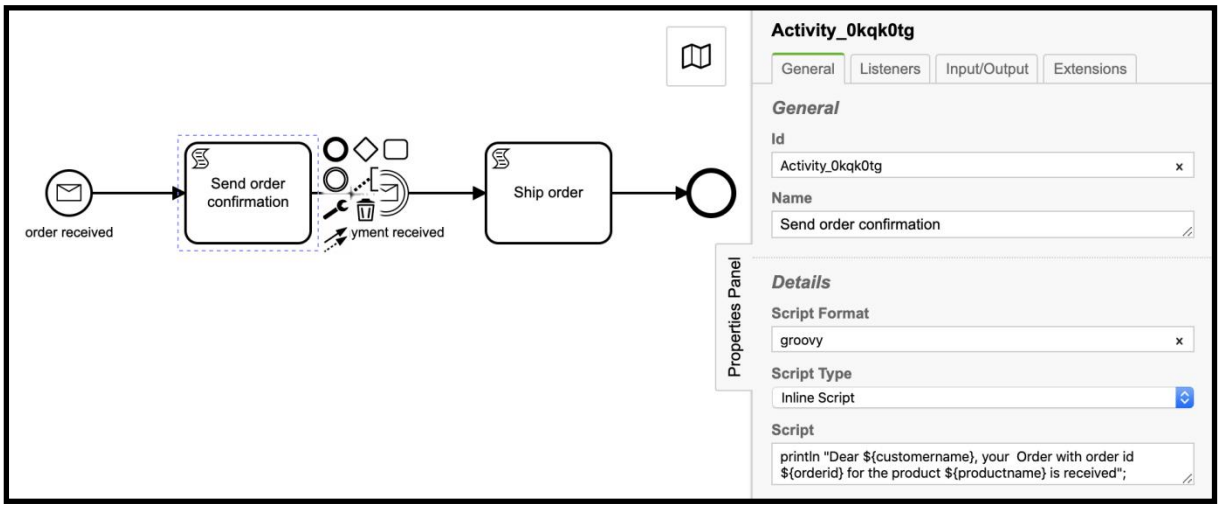
Id: StartEvent_1

Name: order received

Message: Message_order_received (id=Message_0ujcnmu)

Message name: Message_order_received

2.2 Script task: Send order confirmation



Id: StartEvent_1

Name: order received

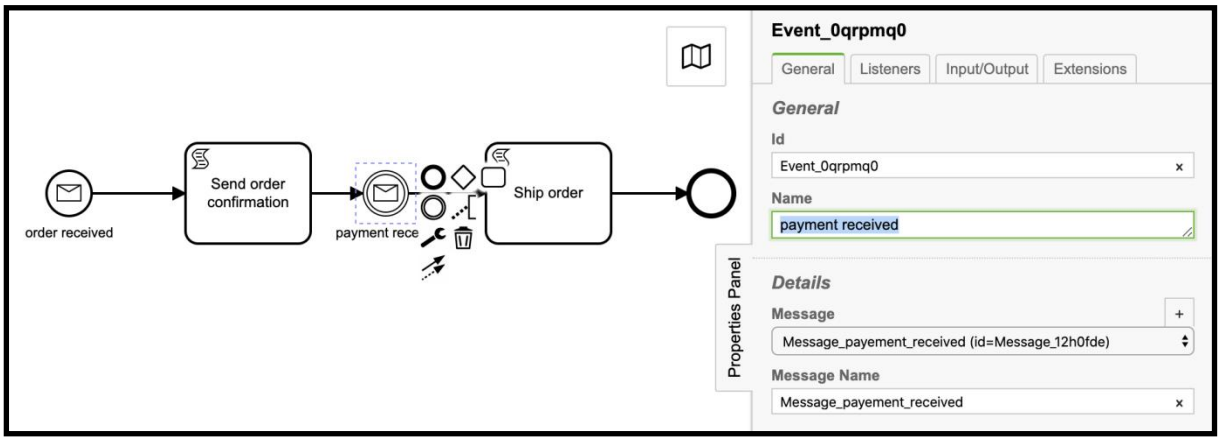
Script Format: groovy

Script Type: Inline Script

Script:

> println "Dear ${customername}, your Order with order id ${orderid} for the product ${productname} is received";

2.3 Message intermediate catch event: payment received



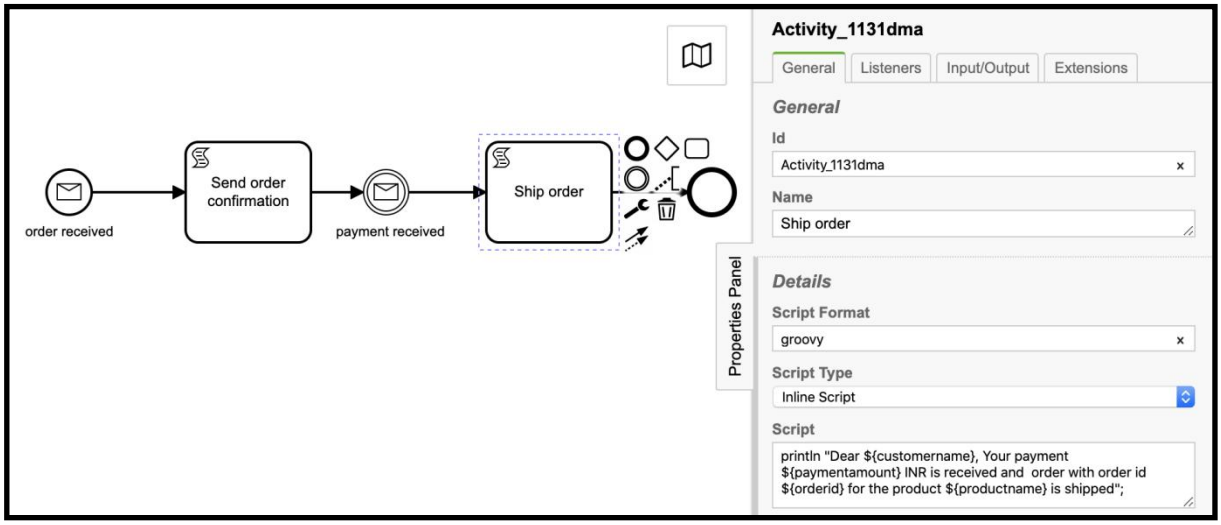
Id: Event_0qrpmq0

Name: payment received

Message: Message_payement_received (id=Message_12h0fde)

Message name: Message_payement_received

2.4 Script task: Send order confirmation



Id: Activity_1131dma

Name: Ship order

Script Format: groovy

Script Type: Inline Script

Script:

```
println "Dear ${customername}, Your payment ${paymentamount} INR is received and order with order id ${orderid} for the product ${productname} is shipped";
```

2.5 CamundaBpmProcessApplication.java

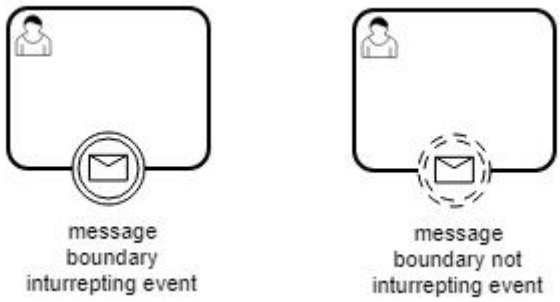
```
package com.camunda.training;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Logger;

import org.camunda.bpm.application.PostDeploy;
import org.camunda.bpm.application.ProcessApplication;
import org.camunda.bpm.application.impl.ServletProcessApplication;
import org.camunda.bpm.engine.ProcessEngine;
import org.camunda.bpm.engine.RuntimeService;
import org.camunda.bpm.engine.runtime.Execution;
import org.camunda.bpm.engine.runtime.ProcessInstance;

/**
 * Process Application exposing this application's resources the process engine.
 */
@ProcessApplication
public class CamundaBpmProcessApplication extends ServletProcessApplication {

	private final Logger LOGGER =
Logger.getLogger(CamundaBpmProcessApplication.class.getName());

	private static final String MESSAGE_ORDER_RECEIVED_ID =
"Message_order_received";
	private static final String MESSAGE_PAYMENT_RECEIVED_ID =
"Message_payement_received";

	/**
	 * In a @PostDeploy Hook you can interact with the process engine and
access the
	 * processes the application has deployed.
	 */
	@PostDeploy
	public void onDeploymentFinished(ProcessEngine processEngine) {
		RuntimeService runtimeService = processEngine.getRuntimeService();
		LOGGER.info("Runtime service started");

		//INTERMEDIATE MESSAGE CATCH EVENT
		int orderId = 1234;
		Map<String, Object> variableMap = new HashMap<String, Object>();
		variableMap.put("customername", "Sumit");
		variableMap.put("orderid", orderId);
		variableMap.put("productname", "MacBook pro");
```

```
                ProcessInstance processInstance =
runtimeService.startProcessInstanceByMessage(MESSAGE_ORDER_RECEIVED_I
D, variableMap);
                LOGGER.info("Process instance started : process instance id : " +
processInstance.getId());

                Execution execution = runtimeService.createExecutionQuery()

          .messageEventSubscriptionName(MESSAGE_PAYMENT_RECEIVED_ID)
                                .processVariableValueEquals("orderid", orderId)
                                .singleResult();

                LOGGER.info("Execution id : "+execution.getId());

                variableMap.put("paymentamount", 175000);

        runtimeService.messageEventReceived(MESSAGE_PAYMENT_RECEIVED
_ID, execution.getId(), variableMap);
                LOGGER.info("Payment received message notified to process engine");


        }

}
```

On deployment of process application

1. process instance will be initiated through message start event
2. Script task Send order confirmation will be executed
3. Intermediate message catch event will wait for message to arrive for payment confirmation.
4. Script task Ship order will get executed and process will be terminated.

3. Message boundary event

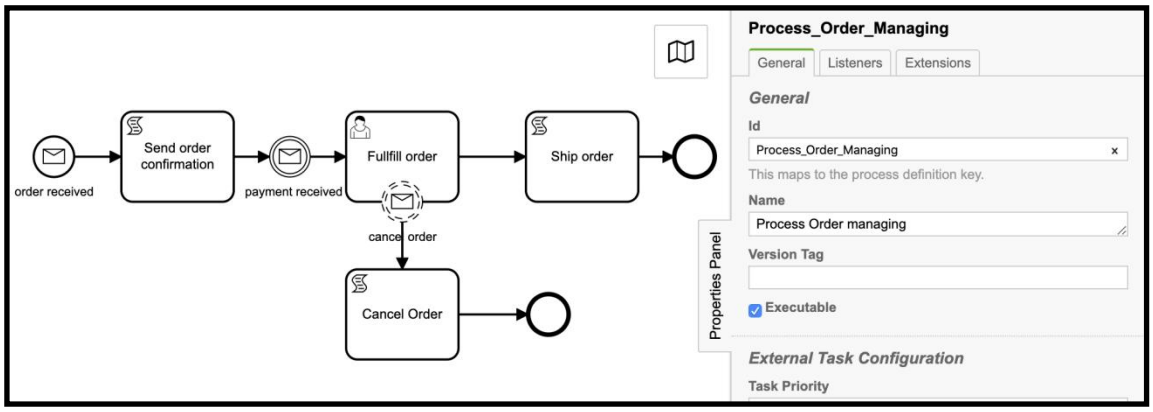


Boundary events are catching events that are attached to an activity. This means that while the activity is running, the message boundary event is listening for named message. When this is caught, two things might happen, depending on the configuration of the boundary event:

- Interrupting boundary event: The activity is interrupted and the sequence flow going out of the event is followed.
- Non-interrupting boundary event: One token stays in the activity and an additional token is created which follows the sequence flow going out of the event.

3.1 Process definition

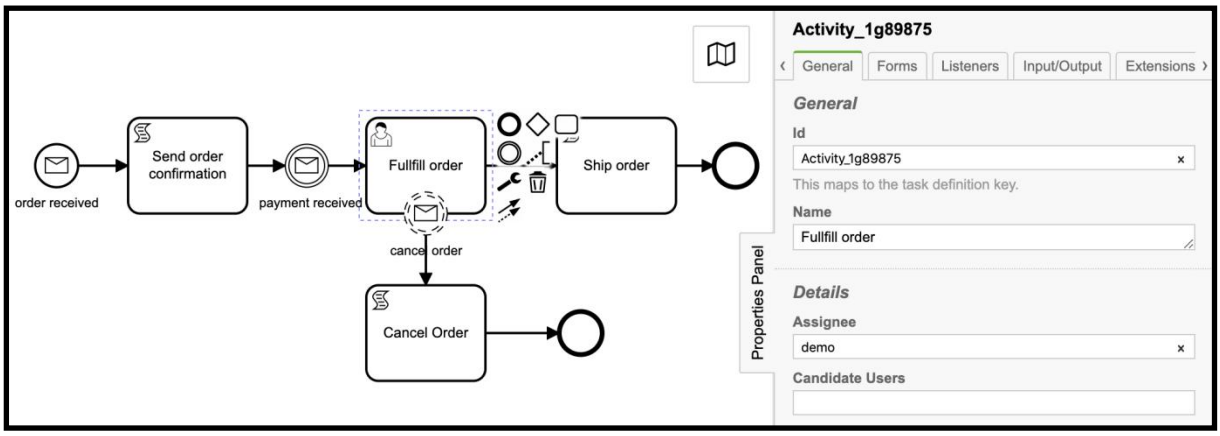
We will demonstrate message boundary event using following process diagram.



Id: Process_Order_Managing

Name: Process Order managing

3.2 User task: Fullfill order

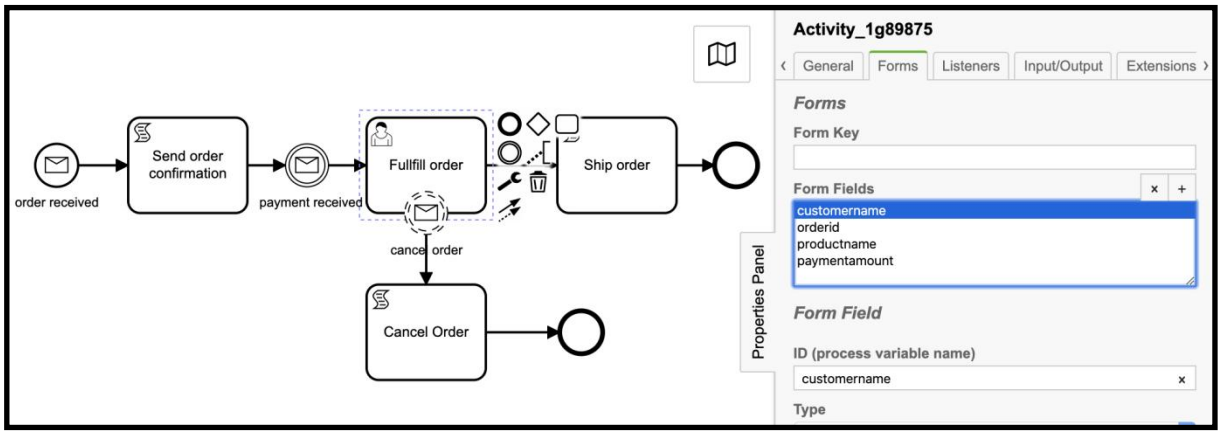


Id: Process_Order_Managing

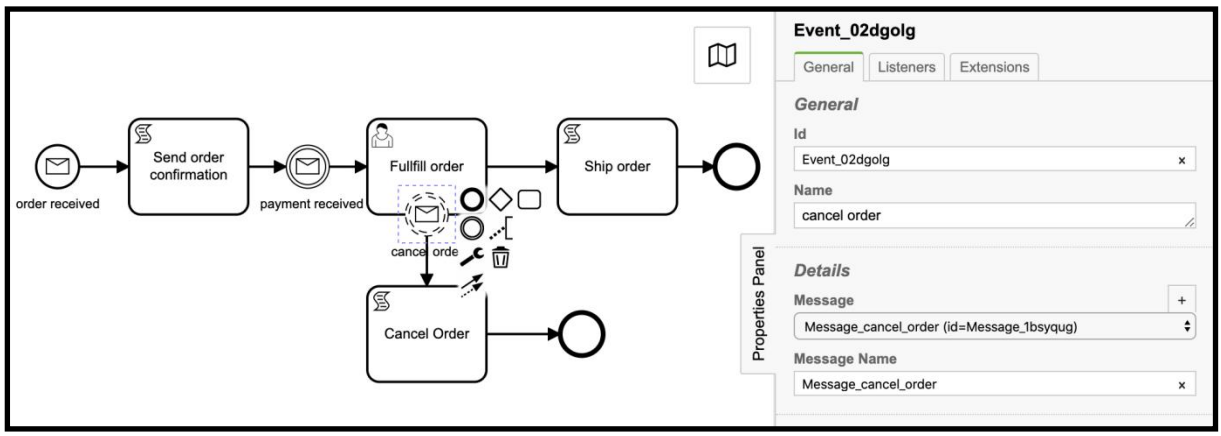
Name: Process Order managing

Assignee: demo

Task form:



3.3 Message boundary event



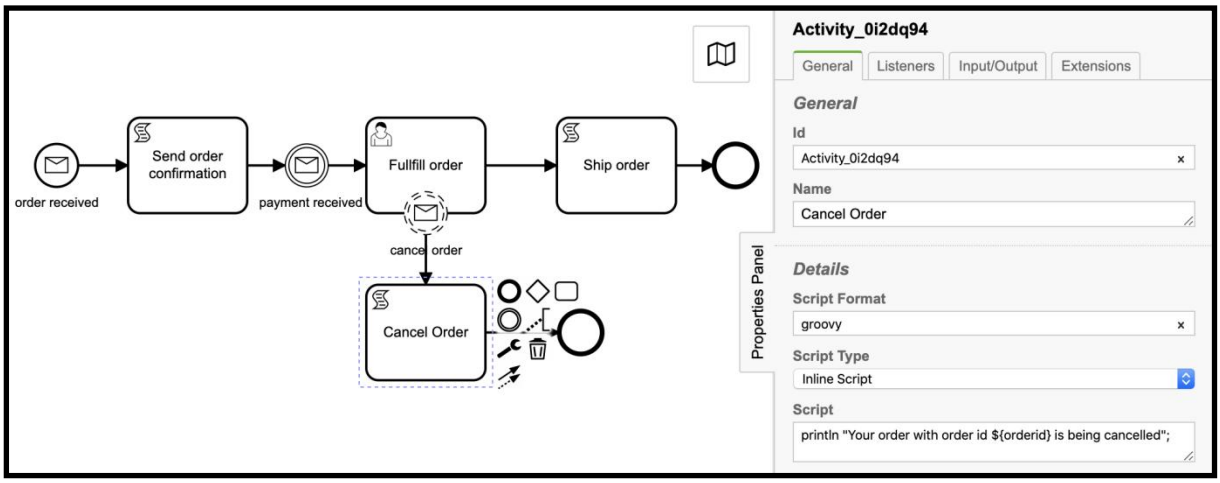
Id: Event_02dgolg

Name: cancel order

Message: Message_cancel_order (id=Message_1bsyqug)

Message name: Message_cancel_order

3.4 Script task: cancel order



Id: Activity_0i2dq94

Name: Cancel Order

Script Format: groovy

Script Type: Inline Script

Script:

```
println "Your order with order id ${orderid} is being cancelled";
```

3.5 CamundaBpmProcessApplication.java

```
package com.camunda.training;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Logger;

import org.camunda.bpm.application.PostDeploy;
import org.camunda.bpm.application.ProcessApplication;
import org.camunda.bpm.application.impl.ServletProcessApplication;
```

```java
import org.camunda.bpm.engine.ProcessEngine;
import org.camunda.bpm.engine.RuntimeService;
import org.camunda.bpm.engine.runtime.Execution;
import org.camunda.bpm.engine.runtime.ProcessInstance;

/**
 * Process Application exposing this application's resources the process engine.
 */
@ProcessApplication
public class CamundaBpmProcessApplication extends ServletProcessApplication {

	private final Logger LOGGER =
Logger.getLogger(CamundaBpmProcessApplication.class.getName());

	private static final String MESSAGE_ID = "Message employee onboard";
	private static final String MESSAGE_ORDER_RECEIVED_ID =
"Message_order_received";
	private static final String MESSAGE_PAYMENT_RECEIVED_ID =
"Message_payement_received";
	private static final String MESSAGE_ORDER_CANCEL_ID =
"Message_cancel_order";

	/**
	 * In a @PostDeploy Hook you can interact with the process engine and
access the
	 * processes the application has deployed.
	 */
	@PostDeploy
	public void onDeploymentFinished(ProcessEngine processEngine) {
		RuntimeService runtimeService = processEngine.getRuntimeService();
		LOGGER.info("Runtime service started");
		// INTERMEDIATE MESSAGE BOUNDARY EVENT
		int orderId = 1234;
		Map<String, Object> variableMap = new HashMap<String, Object>();
		variableMap.put("customername", "Sumit");
		variableMap.put("orderid", orderId);
		variableMap.put("productname", "MacBook pro");

		ProcessInstance processInstance =
runtimeService.startProcessInstanceByMessage("Message_order_received_1",
				variableMap);
		LOGGER.info("Process instance started : process instance id : " +
processInstance.getId());

		Execution execution = runtimeService.createExecutionQuery()

	.messageEventSubscriptionName(MESSAGE_PAYMENT_RECEIVED_ID)
				.processVariableValueEquals("orderid",
orderId).singleResult();
```

```
                LOGGER.info("Execution id : " + execution.getId());

                variableMap.put("paymentamount", 175000);

        runtimeService.messageEventReceived(MESSAGE_PAYMENT_RECEIVED
_ID, execution.getId(), variableMap);
                LOGGER.info("Payment received message notified to process engine");

                execution =
runtimeService.createExecutionQuery().messageEventSubscriptionName(MESSAG
E_ORDER_CANCEL_ID)
                                .processVariableValueEquals("orderid",
orderId).singleResult();


        runtimeService.messageEventReceived(MESSAGE_ORDER_CANCEL_ID,
execution.getId(), variableMap);
                LOGGER.info("Order message cancelled notified to process engine");

        }

}
```

On deployment of process application

1. process instance will be initiated through message start event
2. Script task Send order confirmation will be executed
3. Intermediate message catch event will wait for message to arrive for payment confirmation.
4. User task Fullfill order will get created.
5. Trigger boundary message event to cancel order by API
6. Script task to notify order cancellation will get executed.

4. Message intermediate throw event

A Message Intermediate Throwing event sends a message to an external service. This event has the same behavior as a Service Task.

```
<intermediateThrowEvent id="message">
```

```
  <messageEventDefinition
camunda:class="com.camunda.training.SendMessageDelegate" />
</intermediateThrowEvent>
```

5. Message end event

When process execution arrives at a Message End Event, the current path of execution is ended and a message is sent. The Message End Event has the same behavior as a Service Task.

```
<endEvent id="end">
  <messageEventDefinition
camunda:class="org.camunda.bpm.MyMessageServiceDelegate" />
</endEvent>
```